
Teaching Machines to Code: Neural Markup Generation with Interpretable Attention

Sumeet S. Singh
Independent Researcher
Saratoga, CA 95070
sumeet@singhonline.info

Abstract

We present a neural transducer model with visual attention that learns to generate $\text{\LaTeX}$ markup of a real-world math formula given its image. Applying sequence modeling and transduction techniques that have been very successful across modalities such as natural language, image, handwriting, speech and audio; we construct an image-to-markup model that learns to produce syntactically and semantically correct $\text{\LaTeX}$ markup code over 150 words long and achieves a BLEU score of 89%; improving upon the previous state-of-art for the Im2Latex problem. We also demonstrate with heat-map visualization how attention helps in interpreting the model and can pinpoint (localize) symbols on the image accurately despite having been trained without any bounding box data.

1 Introduction

In the past decade, deep neural network models based on RNNs¹, CNNs² and ‘attention’ [29] have been shown to be very powerful sequence modelers and transducers. Their ability to model joint distributions of real-world data has been demonstrated through remarkable achievements in a broad spectrum of generative tasks such as; image synthesis [27, 28, 22, 25], image description [16, 31, 14, 21, 30], video description [7], speech and audio synthesis [26], handwriting recognition [12, 2], handwriting synthesis [9], machine translation [5, 1, 15, 24], speech recognition [10, 4, 11], etc. [8, 29]

One class of sequence models employ the so-called *encoder-decoder* [5] or *sequence-to-sequence* [24] architecture, wherein an *encoder* encodes a source sequence into feature vectors, which a *decoder* employs to produce the target sequence. The source and target sequences may either belong to the same modality (e.g. in machine translation use-cases) or different modalities (e.g. in image-to-text, text-to-image, speech-to-text); the encoder / decoder sub-models being constructed accordingly. The entire model is trained end-to-end using supervised-learning techniques. In recent years, this architecture has been augmented with an *attention and alignment* model which selects a subset of the feature vectors for decoding. It has been shown to help with longer sequences [1, 19]. Among other things, this architecture has been used for image-captioning [31]. In our work we employ a encoder-decoder architecture with attention, to map images of math formulas into corresponding $\text{\LaTeX}$ markup code. The contributions of this paper are: 1) Solves the Im2Latex problem¹⁰⁰ and improves over the previous best reported BLEU score by 1.27% BLEU, 2) Pushes the boundaries of the neural encoder-decoder architecture with visual attention, 3) Analyses variations of the model and cost function. Specifically we note the changes to the base model [31] and what impact those had on performance, 4) Demonstrates the use of attention visualization for model interpretation and

¹Recurrent Neural Network.

²Convolutional Neural Networks and variants such as dilated CNNs [32].

5) Demonstrates how attention can be used to localize objects (symbols) in an image despite having been trained without bounding box data.

1.1 The IM2LATEX problem

The IM2LATEX Problem is a request for research proposed by OpenAI. The challenge is to build a Neural Markup Generation model that can be trained end-to-end to generate the $\text{\LaTeX}$ markup of a math formula given its image. Data for this problem was produced by rendering single-line real-world $\text{\LaTeX}$ formulas obtained from the KDD Cup 2003 dataset. The resulting grayscale images were used as the input samples while the original markup was used as the label/target sequence. Each training/test sample (Figure 1) is comprised of an input image $\mathbf{x}$ and a cor-

$$S_0 = \sum_l \frac{1}{2\Delta_l^2} \text{Tr} \phi_l^a \phi_{-l}^a + \sum_l \frac{1}{2\epsilon_l^2} \text{Tr} f_l^a f_{-l}^a + \sum_r \frac{1}{g_r} \text{Tr} \bar{\psi}_r^a \psi_r^a.$$

$$\text{\LaTeX} \left\{ \sum_{l=1}^{\infty} \frac{1}{2\Delta_l^2} \text{Tr} \left(\phi_l^a \phi_{-l}^a \right) + \sum_{l=1}^{\infty} \frac{1}{2\epsilon_l^2} \text{Tr} \left(f_l^a f_{-l}^a \right) + \sum_{r=1}^{\infty} \frac{1}{g_r} \text{Tr} \left(\bar{\psi}_r^a \psi_r^a \right) \right\}$$

$$\text{\LaTeX} \left\{ \sum_{l=1}^{\infty} \frac{1}{2\Delta_l^2} \text{Tr} \left(\phi_l^a \phi_{-l}^a \right) + \sum_{l=1}^{\infty} \frac{1}{2\epsilon_l^2} \text{Tr} \left(f_l^a f_{-l}^a \right) + \sum_{r=1}^{\infty} \frac{1}{g_r} \text{Tr} \left(\bar{\psi}_r^a \psi_r^a \right) \right\}$$

Figure 1: A training sample: At the top is the input image $\mathbf{x}$, middle the target sequence $\mathbf{y}$ ($\tau = 145$) and bottom the predicted sequence $\hat{\mathbf{y}}$ ($\tau = 148$). Each space-separated word in $\mathbf{y}$ and $\hat{\mathbf{y}} \in V$

responding target $\text{\LaTeX}$ -sequence $\mathbf{y}$ of length τ . Each word $\mathbf{y}$ of the target sequence, belongs to the vocabulary of the dataset plus two special tokens: beginning-of-sequence $\langle \text{bos} \rangle$ and end-of-sequence $\langle \text{eos} \rangle$. Denoting image dimensions as H_I, W_I and C_I and the vocabulary as a set V of K words, we represent $\mathbf{x} \in \mathbb{R}^{H_I \times W_I \times C_I}$, $V := \{\text{\LaTeX} \text{ tokens}, \langle \text{eos} \rangle, \langle \text{bos} \rangle\}; |V| = K$ and $\mathbf{y} := (\mathbf{y}_1, \dots, \mathbf{y}_\tau); \mathbf{y}_t \in \{1, \dots, K\}$. The task is to generate markup that a $\text{\LaTeX}$ compiler will render back to the original image. Therefore, our model needs to generate syntactically and semantically correct markup, by simply ‘looking’ at the image: i.e. it should jointly model vision and language.

2 Image to markup model

Our model (Figure 2a) has the same basic architecture as [31] (which we call our baseline model) in the way the encoder, decoder and a visual attention interact. However there are significant differences in the sub-models which we notate in the remainder of this paper and in the appendix.

2.1 Encoder

All images are standardized to a fixed size by centering and padding with white pixels. Then they are linearly transformed (whitened) to lie in the range $[-0.5, 0.5]$. A deep CNN then encodes the whitened image into a *visual feature grid* $\hat{\mathbf{A}}$, having $\hat{H} \times \hat{W}$ (i.e. height $\times$ width) *visual feature vectors* $\mathbf{a}_{(\hat{h}, \hat{w})} \in \mathbb{R}^{\hat{D}}$. The visual feature vectors are then concatenated (pooled) together in strides of shape $[S_H, S_W]$; begetting *pooled feature vectors* $\mathbf{a}_{(h, w)} \in \mathbb{R}^D$, where $D = \hat{D} \cdot S_H \cdot S_W$. The resulting feature map $\mathbf{A}$, has a correspondingly shrunken shape $[H, W]$; where $H = \hat{H}/S_H$ and $W = \hat{W}/S_W$.

$$\mathbf{A} := \begin{bmatrix} \mathbf{a}_{(1,1)} & \dots & \mathbf{a}_{(1,W)} \\ \vdots & \vdots & \vdots \\ \mathbf{a}_{(H,1)} & \dots & \mathbf{a}_{(H,W)} \end{bmatrix} \quad (1)$$

Each pooled feature vector can be viewed as a rectangular window into the image, bounded by its receptive field.³ The idea behind this is to partition the image into spatially localized regional encodings and setup a decoder architecture (Section 2.2) that selects/emphasizes only the relevant

³Neighboring regions overlap but each region is distinct overall.

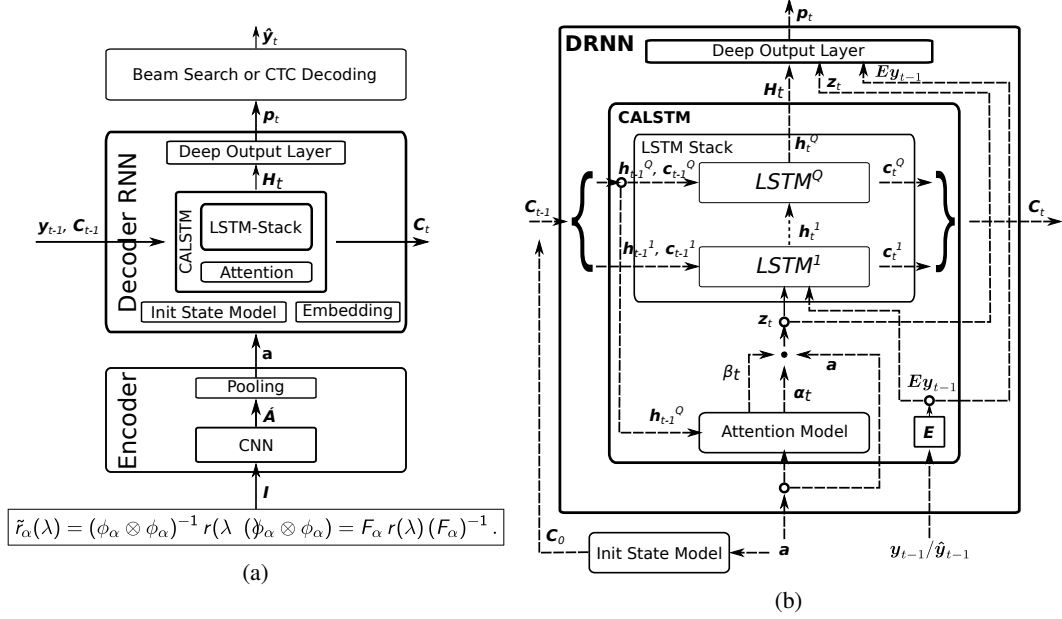


Figure 2: (a) Model outline showing major parts of the model. Beam search decoder is only used during inferencing, not training. LSTM-Stack and Attention model jointly form a Conditioned Attentive LSTM stack (CALSTM) which can itself be stacked. (b) Expanded view of Decoder RNN showing its sub-models. There are three nested RNN cells in all: The decoder RNN (DRNN) at the top level, nesting the CALSTM which nests the LSTM-Stack. The Init Model does not participate in recurrence, therefore its is shown outside the box.

regions at each time-step t , while filtering-out/de-emphasizing the rest. Bahdanau et al. 1 showed that such piecewise encoding enables modeling longer sequences as opposed to models that encode the entire input into a single feature vector [24, 5].⁴ Pooling allows us to construct encoders with different receptive field sizes. We share results of two such models: 12L-NOPOOL with no feature pooling and pooled feature grid shape [4,34] and 12L-STRIPS having stride [4,1] and pooled feature grid shape [1,34]. Finally, for convenience we represent $\hat{A}$ as a flattened sequence $\mathbf{a}$ (Equation 2). See the appendix for more details.

$$\mathbf{a} := (\mathbf{a}_1, \dots, \mathbf{a}_L); \mathbf{a}_l \in \mathbb{R}^D; l = H(h-1) + w; L = HW \quad (2)$$

2.2 Decoder

The decoder is a language modeler and generator. It is a Recurrent Neural Network (DRNN in Figure 2b) that models the discrete probability distribution $\mathbf{p}_t$, of the output word $\mathbf{y}_t$, conditioned on the sequence of previous words $\mathbf{y}_{<t}$ and relevant regions of the encoded image $\mathbf{a}$ ⁵ (Equations 3). Probability of the entire output sequence $\mathbf{y}$ given image $\mathbf{a}$ is therefore given by Equation 4.

$$\begin{aligned} \mathbf{p}_t &: \{1, \dots, K\} \rightarrow [0, 1] \\ \mathbf{y}_t &\sim \mathbf{p}_t \\ \mathbf{p}_t(\mathbf{y}_t) &:= P_r(\mathbf{y}_t | \mathbf{y}_{<t}, \mathbf{a}) \end{aligned} \quad (3)$$

$$P_r(\mathbf{y} | \mathbf{a}) = \prod_{t=1}^{\tau} \mathbf{p}_t(\mathbf{y}_t) \quad (4)$$

⁴That said, Bahdanau et al. 1 employ a bidirectional-LSTM [8] encoder whose receptive field does encompass the entire input anyway! (Although that does not necessarily mean that the bi-LSTM will encode the entire image). Likewise Deng et al. 6 who also solve the IM2LATEX problem also employ a bi-directional LSTM stacked on top of a CNN-encoder in order to get full view of the image. In contrast, our visual feature vectors hold only spatially local information which we found are sufficient to achieve good accuracy. This is probably owing to the nature of the problem; i.e. transcribing a one-line math formula into L^AT_EX sequence requires only local information at each step.

⁵This is now a very standard way to model sequence (sentence) probabilities in neural sequence-generators. See [24] for example.

$$\text{DRNN} : \{\mathbf{a}; \mathbf{y}_{t-1}; \mathbf{C}_{t-1}\} \rightarrow \{\mathbf{p}_t; \mathbf{C}_t\} \quad (5)$$

The DRNN receives the previous word $\mathbf{y}_{t-1}$ and encoded image $\mathbf{a}$ as inputs. In addition, it maintains an internal state $\mathbf{C}_t$ that propagates information (features) extracted from an initial state, the output sequence unrolled thus far and image regions attended to thus far (Equation 5). It is as complex model, comprised of the following sub-models (Figure 2b): 1) A LSTM-Stack [13] responsible for memorizing $\mathbf{C}_t$ and producing a recurrent activation $\mathbf{H}_t$, 2) A Visual attention and alignment model responsible for selecting relevant regions of the encoded image for input to the LSTM-Stack, ⁶ 3) A Deep Output Layer [20] that produces the output probabilities $\mathbf{p}_t$, 4) Init Model: A model that generates the initial state $\mathbf{C}_0$ and 5) An embedding matrix $\mathbf{E}$ (learned by training) that transforms $\mathbf{y}_t$ into a dense representation $\in \mathbb{R}^m$.

2.2.1 Inferencing

After the model is trained, the output sequence is generated by starting with the word ‘bos’ and then repeatedly sampling from $\mathbf{p}_t$ until <eos> is produced. The sequence of words thus sampled is the predicted sequence: $\hat{\mathbf{y}} := (\hat{\mathbf{y}}_1, \dots, \hat{\mathbf{y}}_{\hat{\tau}})$; $\hat{\mathbf{y}}_t \in \mathbb{R}^K$. For this procedure we use beam search decoding [8] with a beam width of 10. Figure 1 shows an example predicted sequence and Figures 5 and 6 show examples of predictions rendered into images by a $\text{\LaTeX}$ compiler.

2.2.2 Visual attention and alignment model

As previously alluded, the decoder soft selects/filters relevant (encoded) image regions at each step. This is implemented via. a ‘soft attention’ mechanism⁷ which computes a weighted sum $\mathbf{z}_t$ of the pooled feature vectors $\mathbf{a}_l$. The visual attention model f_{att} , computes the weight distribution α_t (Equations 6, 7 and 8). f_{att} is modeled by an MLP (details in the appendix). While it is possible for α_t to end up uniformly distributed over $(\mathbf{a}_1 \dots \mathbf{a}_L)$, in practice we see a unimodal shape with most of the weight concentrated on 1-4 neighborhood (see Figure 3) around the mode. We call this neighborhood the *focal-region* - i.e. the focus of attention. In other words we empirically observe that the attention model’s focus is ‘sharp’; converging towards the ‘hard attention’ formulation described by Xu et al. [31]. Also note that (Figure 3), the attention model is able to utilize the extra granularity available to it in the I2L-NOPOOL case and consequently generates much sharper focal-regions than I2L-STRIPS.

$$\alpha_t := (\alpha_{t,1}, \dots, \alpha_{t,L}) \quad \left| \quad \begin{array}{l} 0 \leq \alpha_{t,l} \leq 1 \\ \sum_l \alpha_{t,l} = 1 \end{array} \right. \quad (6)$$

$$\alpha_t = f_{att}(\mathbf{a}; \mathbf{H}_{t-1}) \quad (7)$$

$$\mathbf{z}_t = \alpha_t \mathbf{a}^\top \quad (8)$$

Furthermore, the model aligns the focal-region with the output word and thus scans text on the image left-to-right (I2L-STRIPS) or left-right and up-down (I2L-NOPOOL) just like a person would read it (Figure 3). We also observe that it doesn’t focus on empty margins of the image except at the first and last (<eos>) steps which is quite intuitive for determining the beginning or end of text.

2.2.3 LSTM stack

The core sequence generator of the DRNN is a multilayer LSTM [9] (Figure 2b). Our LSTM cell implementation follows Graves et al. [11]. The LSTM cells are stacked in a multi-layer configuration [33, 20] as in Equation 9. $LSTM^q$ is the LSTM cell at position q with $\mathbf{x}_t^q$, $\mathbf{h}_t^q$ and $\mathbf{c}_t^q$ being its input, hidden activation and cell state respectively. $LSTM^1$ receives the stack’s input: soft attention context $\mathbf{z}_t$ and previous output word $\mathbf{E}\mathbf{y}_{t-1}$. $LSTM^Q$ produces the stack’s output $\mathbf{H}_t = \mathbf{h}_t^Q$, which is sent up to the Deep Output Layer. Accordingly, the stack’s activation ($\mathbf{H}_t$) and state ($\mathbf{C}_t$) are defined as: $\mathbf{H}_t = \mathbf{h}_t^Q$ and $\mathbf{C}_t := (\mathbf{c}_t^1, \dots, \mathbf{c}_t^Q, \mathbf{h}_t^1, \dots, \mathbf{h}_t^Q)$. We

$$\begin{aligned} LSTM^q : \{\mathbf{x}_t^q; \mathbf{h}_{t-1}^q; \mathbf{c}_{t-1}^q\} &\rightarrow \{\mathbf{h}_t^q; \mathbf{c}_t^q\} \\ 1 \leq q \leq Q; \mathbf{h}_t^q, \mathbf{c}_t^q &\in \mathbb{R}^n \\ \mathbf{x}_t^q &= \mathbf{h}_t^{q-1} \quad ; q \neq 1 \\ \mathbf{x}_t^1 &= \{\mathbf{z}_t; \mathbf{E}\mathbf{y}_{t-1}\} \end{aligned} \quad (9)$$

⁶The LSTM-Stack and Visual Attention and Alignment model jointly form a Conditioned Attentive LSTM (CALSTM); $\mathbf{H}_t$ and $\mathbf{C}_t$ being its activation and internal state respectively. Our source-code implements the CALSTM as a RNN cell which may be used as a drop-in replacement for a RNN cell.

⁷‘Soft’ attention as defined by Xu et al. [31] and originally proposed by Bahdanau et al. [1].

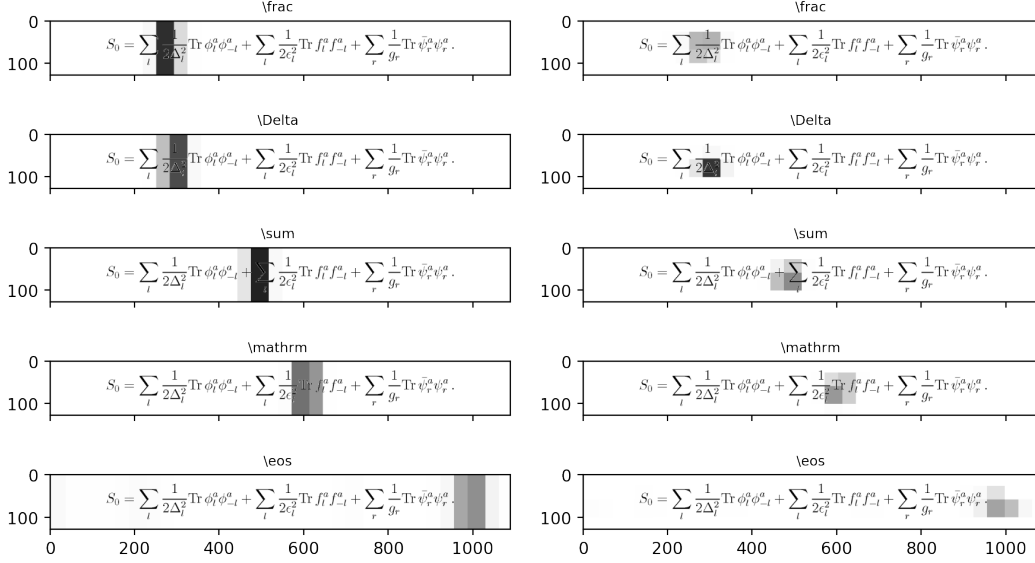


Figure 3: Focal-regions learnt by the attention model: to the left by I2L-STRIPS and to the right by I2L-NOPOOL. Image darkness is proportional to α_t . Notice how α_t concentrates on the image region corresponding to the output word (shown above the image). The `\frac` command starts a fraction, `\mathrm` sets a font and `\eos` is the `<eos>` token.

do not use skip or residual connections between the cells. Both of our models have two LSTM layers with $n = 1500$. Further discussion and details of this model can be found in the appendix.

2.2.4 Deep output layer

We use a Deep Output Layer [20] to produce the final output probabilities: $p_t = f_{out}(\mathbf{H}_t; \mathbf{z}_t; \mathbf{E}\mathbf{y}_{t-1})$. f_{out} is modeled by an MLP. Note that the output layer receives skip connections from the LSTM-Stack input (Equation 9). Details of this model can be found in the appendix.

2.2.5 Init model

The Init Model f_{init} , produces the initial state $\mathbf{C}_0$ of the LSTM-Stack. f_{init} is intended to ‘look’ at the entire image ($\mathbf{a}$) and setup the decoder appropriately before it starts generating the output.

$$f_{init} : \mathbf{a} \rightarrow (\mathbf{c}_0^1, \dots, \mathbf{c}_0^Q, \mathbf{h}_0^1, \dots, \mathbf{h}_0^Q) \quad (10)$$

$$\mathbf{h}_0^q, \mathbf{c}_0^q \in \mathbb{R}^n$$

That said, since it only provides a very small improvement in performance in exchange for over 7 million parameters, its need could be questioned. f_{init} is modeled as an MLP with common hidden layers and $2Q$ distinct output layers, one for each element of $\mathbf{C}_0$, connected as in Figure 4. See the appendix for more detail and discussion.

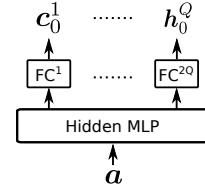


Figure 4: Init Model. FC = Fully Connected Layer.

2.3 Training

The entire model was trained end-to-end by minimizing the objective function $\mathcal{J}$ (Equation 11) using back propagation through time. The first term in Equation 11 is the average (per-word) log perplexity of the predicted sequence⁸ and is the main objective.

$\mathcal{R}$ is the L2-regularization term, equal to L2-norm of the model’s parameters θ (weights and biases)

$$\mathcal{J} = -\frac{1}{\tau} \log(P_r(\mathbf{y}|\mathbf{a})) + \lambda_R \mathcal{R} \quad (11)$$

$$\mathcal{R} = \frac{1}{2} \sum_{\theta} \theta^2 \quad (11a)$$

⁸i.e. Average cross-entropy, negative log-likelihood or negative log-probability.

and λ_R is a hyperparameter requiring tuning. Following Xu et al. [31] at first, we had included a penalty term intended to bias the distribution of the cumulative attention placed on an image-location $\alpha_l := \sum_{t=1}^T \alpha_{t,l}$. However we removed it for various reasons which are discussed in the appendix along with other details and analyses.

We split the dataset into two fixed parts: 1) training dataset = 90-95% of the data and 2) test dataset 5-10%. At the beginning of each run, 5% of the training dataset was randomly held out as the validation-set and the remainder was used for training. Therefore, each such run had a different training/validation data-split, thus naturally cross-validating our learnings across the duration of the project. We trained the model in minibatches of 56 using the ADAM optimizer [17]; periodically evaluating it over the validation set⁹. For efficiency we batched the data such that each minibatch had similar length samples. For the final evaluation however, we fixed the training and validation dataset split and retrained our models for about 100 epochs ($\sim 2\frac{1}{2}$ days). We then picked the model-snapshots with the best validation BLEU score and evaluated the model over the test-dataset for publication. Table 1 lists the training parameters and metrics of various configurations. Training sequence predictions ($\hat{y}$) were obtained by CTC-decoding [10] p_t . Training BLEU score was then calculated over 100 consecutive mini-batches. We used two Nvidia GeForce 1080Ti graphics cards in a parallel towers configuration. Our implementation uses the Tensorflow toolkit and is distributed under AGPL license.

Table 1: Training metrics. $\lambda_R = 0.00005$ and $\beta_2 = 0.9$ for all runs. The number after @ sign is the training epoch of the selected model-snapshot. * denotes that the row corresponds to Table 2.

Dataset	Model	Init Model?	β_1	Train Epochs	Train BLEU	Validation BLEU	Valid'n ED
I2L-140K	I2L-STRIPS	Yes	0.5	104	0.9361	0.8900@72*	0.0677
	I2L-STRIPS	No	0.5	75	0.9300	0.8874@62	0.0691
	I2L-NOPOOL	Yes	0.5	104	0.9333	0.8909@72*	0.0684
	I2L-NOPOOL	No	0.1	119	0.9348	0.8820@92	0.0738
Im2latex-90k	I2L-STRIPS	Yes	0.5	110	0.9366	0.8886@77*	0.0688
	I2L-STRIPS	No	0.5	161	0.9386	0.8810@118	0.0750

3 Results

Given that there are multiple possible $\LaTeX$ sequences that will render the same math image, ideally we should perform a visual evaluation. However, since there is no widely accepted visual evaluation metric, we report corpus BLEU (1,2,3 & 4 grams) and per-word Levenstein Edit Distance¹⁰ scores (see Table 2). We also report a (non-standard) exact visual match score¹⁰³ which reports the percentage of exact visual matches, discarding all partial matches. While the predicted and targeted images match in at least 70%¹⁰³ of the cases, the model generates different but correct sequences (i.e. $y \neq \hat{y}$) in about 40% of the cases (Figure 5). For the cases where the images do not exactly match, the differences in most cases are minor (Figure 6). Overall, our models produce syntactically correct sequences¹¹ for at least 99.85% of the test samples (Table 2). Please visit our website to see hundreds of sample visualizations, analyses and discussions, data-set and source-code.

3.1 Model Interpretability via attention

Since the LSTM stack only sees a filtered view (i.e. focal-region) of the input, it can only base its predictions on the focal-regions seen thus far and initial-state C_0 . Further since the init-model has a negligible impact on performance we can drop it from the model (Table 1) and thereby the

⁹Evaluation cycle was run once or twice per epoch and/or when a training BLEU score calculated on sequences decoded using CTC-Decoding[10] jumped significantly.

¹⁰i.e. Edit distance divided by number of words in the target sequence.

¹¹i.e. Those that were successfully rendered by $\LaTeX 2_{\epsilon}$.

¹⁰³We use the 'match without whitespace' algorithm provided by Deng et al. [6] wherein two images count as matched if they match pixel-wise discarding white columns and allowing for upto 5 pixel image translation (a pdf-latex quirk). It outputs a binary match/no-match verdict for each sample - i.e. partial matches however close, are considered a non-match.

Input Image / Rendered Sequence		y_{len}	$\hat{y}_{len}$
0	$T_{+2-2}^{+q} = \frac{1}{2} \gamma_{q\bar{p}}^i (\Omega_{-2\bar{p}}^{+2i} \psi_{-2\bar{p}}^{1-} - \Omega_{-2}^{+2i} \psi_{+2\bar{p}}^{1-}), T_{+p\pm 2}^{+q} = \frac{1}{2} \gamma_{q\bar{p}}^i \Omega_{+p}^{+2i} \psi_{\pm 2\bar{p}}^{1-},$	147	155
1	$\sigma_{ij}(x^-, y^-, x^+) = \int \frac{dP^-}{4\pi} \frac{dp^+}{4\pi} \frac{dk^+}{4\pi} e^{-\frac{1}{2}P^-x^+} e^{-\frac{1}{2}(p^+x^- - k^+y^-)} \sigma_{ij}(p^+, k^+; P^-),$	150	151
2	$G(f)_{\beta}^{(n)} = \sum_{m=0}^n \{ \tilde{\Theta}_{\beta}^{(n-m)}, \tilde{f}^{(m)} \}_{(q)} + \sum_{m=0}^{(n-2)} \{ \tilde{\Theta}_{\beta}^{(n-m)}, \tilde{f}^{(m+2)} \}_{(\phi)} + \{ \tilde{\Theta}_{\beta}^{(n+1)}, \tilde{f}^{(1)} \}_{(\phi)}$	150	150
3	$S_0 = \sum_l \frac{1}{2\Delta_l^2} \text{Tr} \phi_l^a \phi_{-l}^a + \sum_l \frac{1}{2c_l^2} \text{Tr} f_l^a f_{-l}^a + \sum_r \frac{1}{g_r} \text{Tr} \bar{\psi}_r^a \psi_r^a.$	145	148
4	$ds^2 = -\frac{t^2}{(t^2+r_-^2)(t^2-r_+^2)} dt^2 + t^2(d\phi + \frac{r_+r_-}{t^2} dr)^2 + \frac{(t^2+r_-^2)(t^2-r_+^2)}{t^2} dr^2.$	147	147
5	$H = \frac{1}{2E} U \begin{pmatrix} 0 & 0 & 0 \\ 0 & \Delta m_{21}^2 & 0 \\ 0 & 0 & \Delta m_{31}^2 \end{pmatrix} U^\dagger + \frac{1}{2E} \begin{pmatrix} a & \eta b & 0 \\ \eta^* b & \eta' b & 0 \\ 0 & 0 & 0 \end{pmatrix},$	147	147
6	$D_{\mu\nu}^{ab}(p,p_3) = \frac{\delta^{ab}\delta^{a_3}}{p^2-p_3^2+i\epsilon} \left[-g_{\mu\nu} + p_\mu p_\nu \left((1-\delta_{p_3,0}) \frac{1}{p_3^2} + \delta_{p_3,0} (1-\xi) \frac{1}{p^2+i\epsilon} \right) \right]$	139	145
7	$V(H_1, H_2) = \frac{1}{8} (g_2^2 + g_1^2) (H_1 ^2 - H_2 ^2)^2 + m_1^2 H_1 ^2 + m_2^2 H_2 ^2 - m_3^2 (H_1 H_2 + \text{h.c.})$	144	145
8	$A_0^3(\alpha' \rightarrow 0) = 2g_d \varepsilon_\lambda^{(1)} \varepsilon_\mu^{(2)} \varepsilon_\nu^{(3)} \{ \eta^{\lambda\mu} (p_1^\nu - p_2^\nu) + \eta^{\lambda\nu} (p_3^\mu - p_1^\mu) + \eta^{\mu\nu} (p_2^\lambda - p_3^\lambda) \}.$	146	145
9	$U_L^\dagger M_L U_L = M_L^*, U_L^\dagger M_L U_L = M_L^*, U_L^\dagger M_D U_R = M_D^*, U_R^\dagger M_R U_R = M_R^*.$	149	145
10	$\sqrt{-g} g^{\mu_1 \nu_1} g^{\mu_2 \nu_2} \dots g^{\mu_d - p \nu_d - p} \bar{F}_{\nu_1 \nu_2 \dots \nu_d - p} = \frac{1}{p!} \epsilon^{\mu_1 \mu_2 \dots \mu_d - p \nu_1 \nu_2 \dots \nu_p} F_{\nu_1 \nu_2 \dots \nu_p},$	147	145
11	$\frac{dE}{dz} = \frac{dE_L}{dz} + \frac{dE_{rad}}{dz} \approx \frac{C_2 \alpha_s}{\pi} \mu^2 \ln \frac{3ET}{2\mu^2} \left(\ln \frac{9ET}{\pi^3 T} + \frac{3\pi^2 \alpha_s}{2\mu^2} T^2 \right).$	130	144
12	$L_0 = (2n+1) \frac{ h }{2} + h d_0^\dagger d_0 - \frac{ h }{2} - h \sum_{k=1}^\infty (d_k^\dagger d_k - \tilde{d}_k^\dagger \tilde{d}_k + a_k^\dagger a_k - b_k^\dagger b_k) + L_0^{free}.$	149	144
13	$Q_{\tau\gamma} = \frac{e}{8\pi^2} m_b \bar{q}_\alpha \sigma^{\mu\nu} (1+\gamma_5) b_\alpha F_{\mu\nu}, \quad Q_{8G} = \frac{g}{8\pi^2} m_b \bar{q}_\alpha \sigma^{\mu\nu} t_\alpha^a b_\beta G_{\mu\nu}^a, \quad (q=d \text{ or } s).$	141	143
14	$ds^2 = \alpha' \left(\frac{u^2 h(u)}{R^2} e^{\gamma A} dx_0^2 + \frac{u^2}{R^2} e^{\gamma C} dx_i^2 + \frac{R^2}{u^2 h(u)} e^{\gamma B} du^2 + R^2 e^{\gamma D} d\Omega_5^2 \right),$	143	143
15	$\sin\left(\frac{\tilde{p}_1 \cdot k}{2}\right) \sin\left(\frac{\tilde{p}_2 \cdot k}{2}\right) \sin\left(\frac{\tilde{p}_3 \cdot k}{2}\right) = -\frac{1}{4} (\sin \tilde{p}_1 \cdot k + \sin \tilde{p}_2 \cdot k + \sin \tilde{p}_3 \cdot k)$	133	143
16	$[\mathcal{P}_0, X_0] = i, \quad [\mathcal{P}_i, X_j] = -i \delta_{ij} \left(1 - \frac{P_0^2}{\kappa^2}\right) e^{\mathcal{P}_0/\kappa}, \quad [\mathcal{P}_0, X_i] = -\frac{2i}{\kappa} \mathcal{P}_i e^{\mathcal{P}_0/\kappa}$	139	143
17	$\langle J_{\mu_1}^{a_1}(P_1) \dots J_{\mu_n}^{a_n}(P_n) \rangle_T \approx (-i)^n \frac{N T^2}{12} \delta \Gamma_{\mu_1 \dots \mu_n}^{a_1 \dots a_n}(P_1, \dots, P_n) + O\left(\frac{1}{f_\pi^2}\right),$	143	143
18	$\tilde{u}_\pi(\vec{k}_1)^\dagger \tilde{u}_\pi(\vec{k}_1) = N_\pi^2 \left[\cos^2 \chi(\vec{k}) - \frac{ \vec{p} ^2}{4} (\cos \chi(\vec{k}) c_2(\vec{k}) + \frac{1}{3} \vec{k}^2 b_1(\vec{k})^2) \right].$	142	142
19	$\tau_0(y) = \sum_{\sigma_1=0}^1 \dots \sum_{\sigma_4=0}^1 Y_{\sigma_1 \dots \sigma_4}^{(0)}(t_1 Z_1)^{\sigma_1} (t_2 Z_2)^{\sigma_2} (t_5 Z_5)^{\sigma_3} (t_8 Z_8)^{\sigma_4}$	143	142
20	$\frac{\sin(2\beta)}{32\pi^2} I(\tilde{\Omega}) \rightarrow \frac{\sin(2\beta)}{32\pi^2} (I(\tilde{\Omega}) + c_2^2 c_3^2 \delta_1 I(M_G, M_{G1}) + c_2^2 s_3^2 \delta_2 I(M_G, M_{G2}))$	142	142
21	$\mathcal{W} = Y_e L^j E^c H_1^i \epsilon_{ij} + Y_d Q^{ja} D_a^c H_1^i \epsilon_{ij} + Y_u Q^{ja} U_a^c H_2^i \epsilon_{ij} + \mu H_1^i H_2^j \epsilon_{ij}$	141	141
22	$-i\bar{\kappa}_{(\alpha)} \gamma^\mu \kappa_{(\beta)} \partial_\mu = -i\tilde{C}_{\alpha\gamma} \Gamma_s (T^I)^\gamma{}_\beta \Gamma_{A\text{Adj}}(u^{-1})^\alpha{}_{Ie\alpha} = -i\tilde{C}_{\alpha\gamma} \Gamma_s (T^I)^\gamma{}_\beta k_{(I)},$	145	141

Figure 5: A sample of correct predictions by I2L-STRIPS. We’ve shown the long predictions hence lengths are touching 150. Note that at times the target length is greater than the predicted length and at times the reverse is true (though the original and predicted images were identical). All such cases would evaluate to a less than perfect BLEU score or edit-distance. This happens in about 40% of the cases. For more examples visit our website.

y	$\hat{y}$
0 $\Psi: \tilde{S}^2 \rightarrow \{\mathcal{M}_1, \mathcal{M}_2\}$	$\Psi: \tilde{S}^2 \rightarrow \{\mathcal{M}_1, \mathcal{M}_2\}$
1 $\ln \frac{E + \sqrt{E^2 - m_l^2 - m_\pi}}{E - \sqrt{E^2 - m_l^2 - m_\pi}}$	$\ln \frac{E + \sqrt{E^2 - m_t^2 - m_\pi}}{E - \sqrt{E^2 - m_l^2 - m_\pi}}$
2 $\left(\frac{r_0}{\ell_s}\right)^{\tilde{d}} \sim g_s^{2-k}$	$\left(\frac{r_0}{\ell_s}\right)^{\tilde{d}} \sim g_s^{2-k}$
3 $\dot{p}_a + \epsilon_a^b p_b \omega_\mu \dot{x}^\mu = 0$	$\dot{p}_a + \epsilon_a^b p_b \omega_\mu \dot{x}^\mu = 0$
5 $\hat{T} = \hat{V} + \hat{t} \hat{G} \hat{V},$	$\hat{T} = \hat{V} + \hat{f} \hat{G} \hat{V},$
7 $\Theta^A = (\vartheta^\alpha, \tilde{\vartheta}^\alpha), \quad \partial_A = (\partial_\alpha, \tilde{\partial}^\alpha), \quad \{\partial_A, \Theta^B\} = \delta_A^B$	$\Theta^A = (\vartheta^\alpha, \tilde{\vartheta}^\alpha), \quad \partial_A = (\partial_\alpha, \tilde{\partial}^\alpha), \quad \{\partial_A, \Theta^B\} = \delta_A^B$
8 $\hat{Y}_\tau(M_Z) _{\overline{DR}} = \frac{m_\tau^{pole} - \Re \Sigma_\tau(m_\tau^{pole})}{\bar{v}(M_Z) _{\overline{DR}} \cos \beta(M_Z)} _{\overline{DR}}$	$\hat{Y}_\tau(M_Z) _{\overline{DG}} = \frac{m_\tau^{pole} - \text{kez}_\tau(m_\tau^{pole})}{\delta(M_Z) _{\overline{DG}} \cos \beta(M_Z)} _{\overline{DG}}$
9 $3.4\beta^{[2 2]}(y) = -9y^2 \left[\frac{1-22.21y+36.93y^2}{1-28.21y+143.2y^2} \right]$	$3.4\beta^{[2 2]}(y) = -9y^2 \left[\frac{1-22.2y+36.93y^2}{1-28.2y+143.2y^2} \right]$
10 $\hat{D}^\alpha{}_\beta = (\Pi^2 + 2eBS_3)^\alpha{}_\beta$	$\hat{D}^\alpha{}_\beta = (\Pi^2 + 2eBS_3)^\alpha{}_\beta$
12 $\int \frac{d\vec{z} d\vec{z} d\vec{b} d\vec{b}}{n} \epsilon V(f) = 0$	$\int \frac{d\vec{z} d\vec{z} d\vec{b} d\vec{b}}{n} \epsilon V(f) = 0$
13 $\mathcal{L}_{\Delta S=1}^{(p^4)} = G_8 F^2 \sum_{i=1}^{37} N_i W_i$	$\mathcal{L}_{\Delta S=1}^{(s^0)} = G_8 F^2 \sum_{i=1}^{37} N_i W_i$
14 $\det(\mathcal{M}^{(0)}(N_0)) = \alpha^{Q(0)} \prod_{r,s} [\alpha(h-h_{r,s})]^{P_\ell(N_0-rs/K)},$	$\det(\mathcal{M}^{(0)}(N_0)) = \alpha^{Q(0)} \prod_{r,s} [\alpha(h-h_{r,s})]^{P_f(N_0-rs/K)},$
17 $\gamma \equiv \frac{e^2 N}{2T}.$	$\gamma \equiv \frac{e^2 N}{2T}.$

Figure 6: A random sample of mistakes made by I2L-STRIPS. Observe that usually the model gets most of the formula right and the mistake is only in a small portion of the overall formula (e.g. sample # 1; generating one subscript t instead of an l). In some cases the mistake is in the font and in some cases the images are identical but were incorrectly flagged by the image-match evaluation software (e.g. sample # 0 & #17). In some cases the predicted formula appears more correct than the original! (sample # 10 where position of the subscript β has been ‘corrected’ by I2L-STRIPS).

Table 2: Test results. Im2latex-100k results are from Deng et al. [6]. The last column is the percentage of successfully rendering predictions.

Dataset	Model	BLEU Score	Edit Distance	Visual Match ¹⁰³	Compiling Predictions
I2L-140K	I2L-NOPOOL	89.0%	0.0676	70.37%	99.94%
	I2L-STRIPS	89.0%	0.0671	69.24%	99.85%
Im2latex-90k	I2L-STRIPS	88.19%	0.0725	68.03%	99.81%
Im2latex-100k	IM2TEX	87.73%	-	79.88%	-

dependency on C_0 (now randomly initialized). Therefore if I_t is the focal-region at step t defined by the predicate $\alpha_{t,l} > 0$, then $p_t(\hat{y}_t) = f_L(I_t, I_{t-1} \dots I_0)$ where f_L represents the LSTM-stack and Deep Output Layer. This fact aids considerably in interpreting the predictions of the model. We found heat-map type visuals of the focal-regions (Figure 3) very useful in interpreting the model even as we were developing it.

Object detection via attention: Additionally, we observe that the model settles on a step-by-step alignment of I_t with the output-word’s location on the image: i.e. $p_t(\hat{y}_t) \approx f_L(I_t)$. In other words I_t marks the bounding-box of $\hat{y}_t$ even though we trained without any bounding-box data. Therefore our model -whose encoder has a narrow receptive field- can be applied to the object detection task without requiring bounding box training data, bottom-up region proposals or pretrained classifiers. Note that this is not possible with encoder architectures having wide receptive fields, e.g. those that employ a RNN [6, 1] because their receptive fields encompass the entire input. A future work will quantify the accuracy of object detection [18] using more granular receptive fields. Pedersoli et al. [21] have also used attention for object detection but their model is more complex in that it specifically models bounding-boxes although it doesn’t require them for training.

3.2 Dataset

Datasets were created from single-line L^AT_EX math formulas extracted from scientific papers and subsequently processed as follows: 1) Normalize the formulas to minimize spurious ambiguity.¹² 2) Render the normalized formulas using pdflatex and discard ones that didn’t compile or render successfully. 3) Remove duplicates. 4) Remove formulas with low-frequency words (frequency-threshold = 24 for Im2latex-90k and 50 for I2L-140K). 5) Remove images bigger than 1086×126 and formulas longer than 150. Processing the Im2latex-100k dataset¹⁰⁴ (103559 samples) as above resulted in the **Im2latex-90k** dataset which has 93741 samples. Of these, 4648 were set aside as the test dataset and the remaining 89093 were split into training (95%) and validation (5%) sets before each run (section 2.3). We found the Im2latex-90k dataset too small for good generalization and therefore augmented it with additional samples from KDD Cup 2003. This resulted in the **I2L-140K** dataset with 114406 (training), 14280 (validation) and 14280 (test) samples. Since the normalized formulas are already space separated token sequences, no additional tokenization step was necessary. The vocabulary was therefore produced by simply identifying the set of unique space-separated words in the dataset.

Ancillary material All ancillary material: Both datasets, our model and data-processing source code, visualizations, result samples etc. is available at our website. Appendix is provided alongside this paper.

References

- [1] Bahdanau, D., Cho, K., and Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473.
- [2] Bluche, T. (2016). Joint line segmentation and transcription for end-to-end handwritten paragraph recognition. In *NIPS*.

¹²Normalization was performed using the method and software used by [6] which parses the formulas into an AST and then converts them back to normalized sequences.

¹⁰⁴Im2latex-100k dataset is provided by [6].

- [3] Bluche, T., Ney, H., and Kermorvant, C. (2014). A comparison of sequence-trained deep neural networks and recurrent neural networks optical modeling for handwriting recognition. In *SLSP*.
- [4] Chan, W., Jaitly, N., Le, Q. V., and Vinyals, O. (2015). Listen, attend and spell. *CoRR*, abs/1508.01211.
- [5] Cho, K., van Merriënboer, B., Çaglar Gülçehre, Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *EMNLP*.
- [6] Deng, Y., Kanervisto, A., Ling, J., and Rush, A. M. (2017). Image-to-markup generation with coarse-to-fine attention. In *ICML*.
- [7] Donahue, J., Hendricks, L. A., Guadarrama, S., Rohrbach, M., Venugopalan, S., Saenko, K., and Darrell, T. (2015). Long-term recurrent convolutional networks for visual recognition and description. *CoRR*, abs/1411.4389.
- [8] Graves, A. (2008). Supervised sequence labelling with recurrent neural networks. In *Studies in Computational Intelligence*.
- [9] Graves, A. (2013). Generating sequences with recurrent neural networks. *CoRR*, abs/1308.0850.
- [10] Graves, A., Fernández, S., Gomez, F. J., and Schmidhuber, J. (2006). Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *ICML*.
- [11] Graves, A., Mohamed, A., and Hinton, G. E. (2013). Speech recognition with deep recurrent neural networks. *CoRR*, abs/1303.5778.
- [12] Graves, A. and Schmidhuber, J. (2008). Offline handwriting recognition with multidimensional recurrent neural networks. In *NIPS*.
- [13] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Comput.*, 9(8):1735–1780.
- [14] Johnson, J., Karpathy, A., and Fei-Fei, L. (2016). Denscap: Fully convolutional localization networks for dense captioning. *CoRR*, abs/1511.07571.
- [15] Kalchbrenner, N., Espeholt, L., Simonyan, K., van den Oord, A., Graves, A., and Kavukcuoglu, K. (2016). Neural machine translation in linear time. *CoRR*, abs/1610.10099.
- [16] Karpathy, A. and fei Li, F. (2015). Deep visual-semantic alignments for generating image descriptions. *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3128–3137.
- [17] Kingma, D. P. and Ba, J. (2014). Adam: A method for stochastic optimization. *CoRR*, abs/1412.6980.
- [18] Liu, C., Mao, J., Sha, F., and Yuille, A. L. (2017). Attention correctness in neural image captioning. In *AAAI*.
- [19] Luong, M., Pham, H., and Manning, C. D. (2015). Effective approaches to attention-based neural machine translation. *CoRR*, abs/1508.04025.
- [20] Pascanu, R., Çaglar Gülçehre, Cho, K., and Bengio, Y. (2013). How to construct deep recurrent neural networks. *CoRR*, abs/1312.6026.
- [21] Pedersoli, M., Lucas, T., Schmid, C., and Verbeek, J. (2016). Areas of attention for image captioning. *CoRR*, abs/1612.01033.
- [22] Salimans, T., Karpathy, A., Chen, X., and Kingma, D. P. (2017). Pixelcnn++: Improving the pixelcnn with discretized logistic mixture likelihood and other modifications. *CoRR*, abs/1701.05517.
- [23] Simonyan, K. and Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *CoRR*, abs/1409.1556.

- [24] Sutskever, I., Vinyals, O., and Le, Q. V. (2014). Sequence to sequence learning with neural networks. *CoRR*, abs/1409.3215.
- [25] Theis, L. and Bethge, M. (2015). Generative image modeling using spatial lstms. In *NIPS*.
- [26] van den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., Kalchbrenner, N., Senior, A. W., and Kavukcuoglu, K. (2016a). Wavenet: A generative model for raw audio. *CoRR*, abs/1609.03499.
- [27] van den Oord, A., Kalchbrenner, N., and Kavukcuoglu, K. (2016b). Pixel recurrent neural networks. In *ICML*.
- [28] van den Oord, A., Kalchbrenner, N., Vinyals, O., Espeholt, L., Graves, A., and Kavukcuoglu, K. (2016c). Conditional image generation with pixelcnn decoders. *CoRR*, abs/1606.05328.
- [29] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *NIPS*.
- [30] Vinyals, O., Toshev, A., Bengio, S., and Erhan, D. (2015). Show and tell: A neural image caption generator. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [31] Xu, K., Ba, J., Kiros, J. R., Cho, K., Courville, A. C., Salakhutdinov, R., Zemel, R. S., and Bengio, Y. (2015). Show, attend and tell: Neural image caption generation with visual attention. In *ICML*.
- [32] Yu, F. and Koltun, V. (2015). Multi-scale context aggregation by dilated convolutions. *CoRR*, abs/1511.07122.
- [33] Zaremba, W., Sutskever, I., and Vinyals, O. (2014). Recurrent neural network regularization. *CoRR*, abs/1409.2329.

A Qualitative analyses and details

This section is an appendix to the paper. We present here further details, analyses and discussion of our experiments and comparison with related work.

A.1 Encoder

Table 3 shows the configuration of the Encoder CNN. All convolution kernels have shape (3,3), stride (1,1) and *tanh* non-linearity, whereas all maxpooling windows have shape (2,2) and stride (2,2). We initially experimented with the output of the VGG16 model [23] - per Xu et al. [31]. However

Table 3: Specification of the Encoder CNN.

Layer	Output Shape	Channels
Input (Image)	128×1088	1
Convolution	128×1088	64
Maxpool	64×544	64
Convolution)	64×544	128
Maxpool	32×272	128
Convolution	32×272	256
Maxpool	16×136	256
Convolution	16×136	512
Maxpool	8×68	512
Convolution	8×68	512
Maxpool	$4 \times 34 = (\hat{H} \times \hat{W})$	$512 = (\hat{D})$

(presumably since VGG16 was trained on a different dataset and a different problem) the BLEU score didn't improve beyond 40%. Then we started training VGG16 along with our model but the end-to-end model didn't even start learning (the log-loss curve was flat) - possibly due to the large overall depth of the end-to-end model. Reducing the number of convolution layers to 6 and changing

y	$\hat{y}$
0 $\frac{\partial A_{0\mu}}{\partial t} = -i[A_{0\mu}, H_{F0}]$,	$\frac{\partial A_{0\mu}}{\partial t} = -i[A_{0\mu}, H_{F0}]$,
1 $\{\Phi^i(x), \Phi^j(y)\} = \epsilon^{ij} \delta^2(x-y)$.	$\{\Phi^i(x), \Phi^j(y)\} = \epsilon^{ij} \delta^2(x-y)$.
2 $V_{total} = \sum_i \left \frac{\partial W}{\partial z_i} \right ^2 + V_D + V_{soft}$	$V_{total} = \sum_i \left \frac{\partial W}{\partial z_i} \right ^2 + V_D + V_{soft}$
3 $\alpha_\lambda^{\dagger\alpha}(p) = \int d^3x e^{-ip \cdot x} \left[e^\lambda \cdot (\omega A^a - iE^a) + \int_\Omega (f_1 \Pi^a + f_2 \phi^a) \right]$	$\alpha_\lambda^{\dagger\alpha}(p) = \int d^3x e^{-ip \cdot x} \left[e^\lambda \cdot (\omega A^a - iE^a) + \int_\Omega (f_1 \Pi^a + f_2 \phi^a) \right]$
4 $H_{stat}(k) = P + \frac{i}{vk+i\epsilon}$,	$H_{stat}(k) = P + \frac{i}{vk+i\epsilon}$,
5 $(\phi^* P_s + \phi P_s^*) \frac{2\Delta^2}{M^2}$,	$(\phi^* P_s + \phi P_s^*) \frac{2\Delta^2}{M^2}$,
6 $H_{G/H} = \frac{1}{2} \left(\pi_\alpha - \frac{i\hbar}{2} \Gamma_\alpha \right) g^{\alpha\beta} \left(\pi_\beta + \frac{i\hbar}{2} \Gamma_\beta \right) = \frac{1}{2} \pi_\alpha g^{\alpha\beta} \pi_\beta + V_{G/H}$,	$H_{G/H} = \frac{1}{2} \left(\pi_\alpha - \frac{i\hbar}{2} \Gamma_\alpha \right) g^{\alpha\beta} \left(\pi_\beta + \frac{i\hbar}{2} \Gamma_\beta \right) = \frac{1}{2} \pi_\alpha g^{\alpha\beta} \pi_\beta + V_{G/H}$,
7 $S[\Phi] = S[\phi] + S[\varphi] + S_{\text{int}}[\phi, \varphi]$	$S[\Phi] = S[\phi] + S[\varphi] + S_{\text{int}}[\phi, \varphi]$
8 $\gamma_1 = \frac{\kappa}{4\pi} \quad \gamma_2 = \frac{\lambda}{4}$	$\gamma_1 = \frac{\kappa}{4\pi} \quad \gamma_2 = \frac{\lambda}{4}$
9 $\Gamma = \frac{1-r^2}{8\pi M_B} (M^S ^2 + M^P ^2)$.	$\Gamma = \frac{1-r^2}{8\pi M_B} (M^S ^2 + M^P ^2)$.
10 $E(r) = - \left(\frac{2GE_\nu m_2}{r} \pm \frac{q_1 q_2}{r} + \frac{m_\nu S_1 S_2}{r} \right)$	$E(r) = - \left(\frac{2GE_\nu m_2}{r} \pm \frac{q_1 q_2}{r} + \frac{m_\nu S_1 S_2}{r} \right)$
11 $\chi(x_1, x_2) = \langle 0 T \psi(x_1) \bar{\psi}(x_2) P \rangle$.	$\chi(x_1, x_2) = \langle 0 T \psi(x_1) \bar{\psi}(x_2) P \rangle$.
12 $\epsilon L^{(2)} \theta = \tilde{h}_1^{(2)} v^2 \phi^j (\epsilon \gamma^i \theta) (\theta \gamma^{ij} \theta) + h_1^{(2)} v^i v^j \phi^k (\epsilon \gamma^i \theta) (\theta \gamma^{jk} \theta)$,	$\epsilon L^{(2)} \theta = \tilde{h}_1^{(2)} v^2 \phi^j (\epsilon \gamma^i \theta) (\theta \gamma^{ij} \theta) + h_1^{(2)} v^i v^j \phi^k (\epsilon \gamma^i \theta) (\theta \gamma^{jk} \theta)$,
13 $\sin^2 2\vartheta = \sin^2 2\vartheta_{\text{sun}} = \frac{4 U_{e1} ^2 U_{e2} ^2}{(U_{e1} ^2 + U_{e2} ^2)^2}$.	$\sin^2 2\vartheta = \sin^2 2\vartheta_{\text{sun}} = \frac{4 U_{e1} ^2 U_{e2} ^2}{(U_{e1} ^2 + U_{e2} ^2)^2}$.
14 $F_1 = \frac{g^2}{192\pi^{5/2}} M_{Pl} \simeq 1.5 \times 10^{15} \text{ GeV}$.	$F_1 = \frac{g^2}{192\pi^{5/2}} M_{Pl} \simeq 1.5 \times 10^{15} \text{ GeV}$.
15 $b_{<i>} = \prod_{0 \leq p < q \leq p_i} X_{\tau p(i) \tau q(i)}(z, z)$.	$b_{<i>} = \sum_{0 \leq p < q \leq p} X_{r(i) i \tau(i)}(z, z)$.
16 $F_1^{W+D}(x) = [d^p(x) + \bar{u}^p(x) + d^n(x) + \bar{u}^n(x) + 2s(x) + 2\bar{c}(x)]/2$.	$F_1^{W+D}(x) = [d^p(x) + \bar{u}^p(x) + d^n(x) + \bar{u}^n(x) + 2s(x) + 2\bar{c}(x)]/2$.
17 $u = q^2 b^2/2 \text{ or } Q = Q_0 u, \quad Q_0 = \frac{1}{Am_N b^2}$	$u = q^2 b^2/2 \text{ or } Q = Q_0 u, \quad Q_0 = \frac{1}{Am_N b^2}$
18 $\bar{h}_v \Gamma h_v = \frac{1}{2} \text{Tr} \left(\Gamma P_v \right) \bar{h}_v h_v - \frac{1}{2} \text{Tr} \left(\gamma_\mu \gamma_5 P_v \Gamma P_v \right) \bar{h}_v \gamma^\mu \gamma_5 h_v$,	$\bar{h}_v \Gamma h_v = \frac{1}{2} \text{Tr} \left(\Gamma P_v \right) \bar{h}_v h_v - \frac{1}{2} \text{Tr} \left(\gamma_\mu \gamma_5 P_v \Gamma P_v \right) \bar{h}_v \gamma^\mu \gamma_5 h_v$,
19 $\bar{P}_g(z) = \Delta_{ns} \frac{1}{z} + \frac{1}{1-z}$.	$\bar{P}_g(z) = \Delta_n \frac{1}{z} + \frac{1}{1-z}$.
20 $S \leq S_H, \quad T \geq T_H, \quad E_c \leq E_{BH}, \text{ for } HR \geq 1$	$S \leq S_H, \quad T \geq T_H, \quad E_c \leq E_{BH}, \text{ for } HR \geq 1$
21 $\Gamma \sim N^{-1/2} 10^{23} s^{-1} \exp \left[-\frac{8\sqrt{2}}{3 \cdot 137} \left(\frac{-E}{m_e} \right)^{3/2} \frac{B_0}{NB} A^{1/2} \left(\frac{m_p}{m_e} \right)^{1/2} \right]$,	$\Gamma \sim N^{-1/2} 10^{23} e^{-1} \exp \left[-\frac{8\sqrt{2}}{3 \cdot 137} \left(\frac{-E}{m_e} \right)^{3/2} \frac{B_0}{NB} A^{1/2} \left(\frac{m_p}{m_e} \right)^{1/2} \right]$,
22 $\frac{2 J ^2}{m_0^2} \cong \frac{m_b^2}{m_d^2} \sim 2.5 \times 10^5$.	$\frac{2 J ^2}{m_0^2} \cong \frac{m_b^2}{m_d^2} \sim 2.5 \times 10^5$.
23 $u = \frac{z}{\ell} U^{-1/2} \frac{\partial}{\partial t}$,	$u = \frac{z}{\ell} U^{-1/2} \frac{\partial}{\partial t}$,
24 $\Omega = \frac{\rho_c}{\rho_c}$	$\Omega = \frac{\rho_c}{\rho_c}$
25 $e^{(2r+1)\pi i L(0)} Y_1(v,x) e^{-(2r+1)\pi i L(0)} = Y_1((-1)^{L(0)} v, -x)$,	$e^{(2r+1)\pi i L(0)} Y_1(v,x) e^{-(2r+1)\pi i L(0)} = Y_1((-1)^{L(0)} v, -x)$,
26 $\mathbf{A}_2 = \int d^2x \, \mathcal{A}_2(x) * \delta \alpha(x)$,	$\mathbf{A}_2 = \int d^2x \, \mathcal{A}_2(x) * \delta \alpha(x)$,
27 $ds^2 = (k + f_0 \frac{R_0^2}{R^2})^{-1} dR^2 + R^2 d\Omega_k^2 - (k + f_0 \frac{R_0^2}{R^2}) [dx^5 + A_R(R) dR]^2$	$ds^2 = (k + f_0 \frac{R_0^2}{R^2})^{-1} dR^2 + R^2 d\Omega_k^2 - (k + f_0 \frac{R_0^2}{R^2}) [dx^5 + A_R(R) dR]^2$
28 $[\hat{\rho}_0, \hat{\rho}_0] = 0, \quad [\hat{S}_0^A,$	$[\hat{\rho}_0, \hat{\rho}_0] = 0, \quad [\hat{S}_0^A,$
29 $\mathcal{L}_4 = (F^1 - \partial_5 A^2) \frac{dW}{dA^1} + \dots$.	$\mathcal{L}_4 = (F^1 - \partial_5 A^2) \frac{dW}{dA^1} + \dots$.
30 $\Psi_j \bar{\Psi}_i = \delta_{ij} - q^{-1} \hat{\mathcal{R}}_{iklj} \bar{\Psi}_l \Psi_k$	$\Psi_j \bar{\Psi}_i = \delta_{ij} - q^{-1} \hat{\mathcal{R}}_{iklj} \bar{\Psi}_l \Psi_k$
31 $\Pi_i = 0, \quad \Theta_k = \partial_k \Pi_0 + cm^2 h_k = 0$,	$\Pi_i = 0, \quad \Theta_k = \partial_k \Pi_0 + cm^2 h_k = 0$,
32 $\sin \delta = \frac{s_{23} c_{23}}{s_2 c_2} \sin \delta_{13}$	$\sin \delta = \frac{s_{23} c_{23}}{s_2 c_2} \sin \delta_{13}$
33 $\Delta_A = -2H s_{AA}$,	$\Delta_A = -2H s_{AA}$,
34 $\hat{D}_{\mu\nu}^{-1} = \hat{D}_{\mu\alpha}^{-1} \eta^{\alpha\beta} \left(\eta_{\beta\nu} + \sum_{n=1}^{\infty} A_n (D^{-1})_{\beta\nu}^n \right)$,	$\hat{D}_{\mu\nu}^{-1} = \hat{D}_{\mu\alpha}^{-1} \eta^{\alpha\beta} \left(\eta_{\beta\nu} + \sum_{n=1}^{\infty} A_n (D^{-1})_{\beta\nu}^n \right)$,
35 $H_G(x^2) = -\frac{1}{8\pi G e^2} \left([B(x^2)]^{-2} - 1 \right)$,	$H_G(x^2) = -\frac{1}{8\pi G e^2} \left([B(x^2)]^{-2} - 1 \right)$,
36 $T_{\mu\nu} = T_{\mu\nu}^+ + \frac{a^2}{a^2} T_{\mu\nu}^-$.	$T_{\mu\nu} = T_{\mu\nu}^+ + \frac{a^2}{a^2} T_{\mu\nu}^-$.
37 $G_{H^{d+1}} = \frac{l^{1-d}}{\Sigma_d} \int_q^{+\infty} \frac{dx}{\sinh^d x}$.	$G_{H^{d+1}} = \frac{l^{1-d}}{\Sigma_d} \int_q^{+\infty} \frac{dx}{\sinh^d x}$.
38 $\Gamma_{\{\mu\}}^{(n)} = \frac{\delta A_{\mu_1}^{(n)}(x_1) \cdots \delta A_{\mu_j}^{(n)}(x_j) \cdots \delta A_{\mu_n}^{(n)}(x_n)}{\delta A_{\mu_1}^{(n)}(x_1) \cdots \delta A_{\mu_j}^{(n)}(x_j) \cdots \delta A_{\mu_n}^{(n)}(x_n)}$,	$\Gamma_{\{\mu\}}^{(n)} = \frac{\partial^{n(n)} \Gamma(A')}{\delta A_{\mu_1}^{(n)}(x_1) \cdots \delta A_{\mu_j}^{(n)}(x_j) \cdots \delta A_{\mu_n}^{(n)}(x_n)}$,
39 $\dot{\pi}_{\text{ab}} =$	$\dot{\pi}_{\text{ab}} =$
40 $\gamma \pi_a^0 = 0, \gamma \pi_a^i = f_{ac}^b \pi_b^i \eta_2^c, \gamma \pi_{0i} = 0, \gamma \pi_{ij} = 0,$	$\gamma \pi_a^0 = 0, \gamma \pi_a^i = f_{ac}^b \pi_b^i \eta_2^c, \gamma \pi_{0i} = 0, \gamma \pi_{ij} = 0,$
41 $ Z_1 ^2 = Z_2 ^2 = \frac{1}{(4G)^2} e^{-\eta_0} [(Q_R^1)^2 + (Q_R^2)^2]$,	$ Z_1 ^2 = Z_2 ^2 = \frac{1}{(4G)^2} e^{-\eta_0} [(Q_R^1)^2 + (Q_R^2)^2]$,

Figure 7: A random sample of predictions of I2L-STRIPS containing both good and bad predictions. Note that though this is a random sample, prediction mistakes are not obvious and it takes some effort to point them out! For more examples visit our website.¹⁰⁵

the non-linearity to $\tanh$ (to keep the activations in check) got us good results. Further reducing number of layers to 5 yielded the same performance, therefore we stuck with that configuration (Table 3). In addition, we experimented with I2L-STRIPS because it reduces the rectangular image-map to a linear map, thereby presumably making the alignment model’s task easier because now it would only need to scan in one-dimension. However, it performed around the same as I2L-NOPOOL and therefore that hypothesis was debunked. In fact we prefer I2L-NOPOOL since it has fewer parameters and its attention model has sharper focal-regions which helps with model interpretation.

A.2 Attention model

Table 4 specifies the configuration of the attention model MLP. Xu et al. 31’s formulation of attention model ($\alpha_{t,l} = \text{MLP}(\mathbf{a}_l; \mathbf{H}_{t-1})$) receives inputs from only a single image location. In comparison, our formulation ($\alpha_t = f_{att}(\mathbf{a}; \mathbf{H}_{t-1})$) receives the full encoded image $\mathbf{a}$ in its input. This change was needed because the previous formulation did not progress beyond a point, presumably because this problem warranted a wider receptive field. The new formulation works equally well with different pooling strides (and correspondingly different values of L).

Table 4: Specification of the Visual Attention Model MLP. $L = 34$ for I2L-STRIPS and 136 for I2L-NOPOOL.

Layer	Num Units	Activation
3 (output)	L	softmax
2	$\max(128, L)$	tanh
1	$\max(256, L)$	tanh

Also, Xu et al. 31’s formulation of $\mathbf{z}_t = \beta_t \cdot \alpha_t \cdot \mathbf{a}$ includes a scalar $\beta_t = \text{MLP}(\mathbf{H}_{t-1})$ which informs the LSTM how much emphasis to place on the image v/s the language model. Experimentally we found that it had no impact on end-to-end performance, therefore we dropped it from our model.

Xu et al. 31 also use a simpler formula for $\mathcal{A} = \sum_{l=1}^L (\sum_{t=1}^{\tau} \alpha_{t,l} - 1)^2$ which they call ‘doubly stochastic optimization’. Our formulation uses the true mean of α_l , τ/L instead of 1, normalizes it to a fixed range so that it can be compared across models and more importantly, includes a target-ASE term ASE_T . Without this term, i.e. with $ASE_T = 0$, $\mathcal{A}$ would bias the attention model towards uniformly scanning all the L image locations. This is undesirable since there are many empty regions of the images where it makes no sense for the attention model to spend much time. Conversely, there are some densely populated regions (e.g. a symbol with complex superscript and subscripts) where the model would reasonably spend more time because it would have to produce a longer output sequence. In other words, the optimal scanning pattern would have to be non-uniform - $ASE_T \neq 0$. Also, the scanning pattern would vary from sample to sample, but ASE_T is set to a single value (even if zero) for all samples. Therefore we preferred to remove the attention-model bias altogether from the objective function by setting $\lambda_A = 0$ in all situations except when the attention model needed a ‘nudge’ in order to ‘get off the ground’. In such cases we set ASE_T based on observed values of ASE_N (Table 8).

A.3 LSTM stack

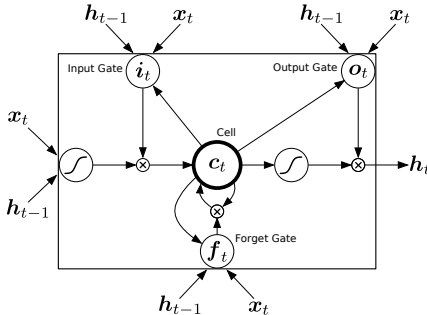


Figure 8: LSTM Cell

$$\begin{aligned}
 \mathbf{i}_t &= \sigma(W_{xi}\mathbf{x}_t + W_{hi}\mathbf{h}_{t-1} + W_{ci}\mathbf{c}_{t-1} + \mathbf{b}_i) \\
 \mathbf{f}_t &= \sigma(W_{xf}\mathbf{x}_t + W_{hf}\mathbf{h}_{t-1} + W_{cf}\mathbf{c}_{t-1} + \mathbf{b}_f) \\
 \mathbf{c}_t &= \mathbf{f}_t \mathbf{c}_{t-1} + \mathbf{i}_t \tanh(W_{xc}\mathbf{x}_t + W_{hc}\mathbf{h}_{t-1} + \mathbf{b}_c) \\
 \mathbf{o}_t &= \sigma(W_{xo}\mathbf{x}_t + W_{ho}\mathbf{h}_{t-1} + W_{co}\mathbf{c}_t + \mathbf{b}_o) \\
 \mathbf{h}_t &= \mathbf{o}_t \tanh(\mathbf{c}_t) \\
 \mathbf{i}_t, \mathbf{f}_t, \mathbf{o}_t, \mathbf{c}_t, \mathbf{h}_t &\in \mathbb{R}^n
 \end{aligned} \tag{12}$$

Our LSTM cell implementation (Figure. 8 and equation 12) follows Graves et al. [11], Zaremba et al. [33]. In equation 12 σ is the logistic sigmoid function and $\mathbf{i}_t$, $\mathbf{f}_t$, $\mathbf{o}_t$, $\mathbf{c}_t$ and $\mathbf{h}_t$ are respectively the *input gate*, *forget gate*, *output gate*, *cell* and *hidden* activation vectors of size n .

During experimentation our penultimate LSTM-stack which had 3 LSTM layers with 1000 units each, gave us a validation score of 87.45%. At that point experimental observations suggested that the LSTM stack was the accuracy 'bottleneck' because other sub-models were performing very well. Increasing the number of LSTM units to 1500 got us better validation score - but a worse overfit. Reducing the number of layers down to 2 got us the best overall validation score. In comparison, Xu et al. [31] have used a single LSTM layer with 1000 cells.

A.4 Deep output layer

Note that the output layer receives skip connections from the LSTM-Stack input ($\mathbf{p}_t = f_{out}(\mathbf{H}_t; \mathbf{z}_t; \mathbf{E}\mathbf{y}_{t-1})$). We observed a 2% impact on the BLEU score with the addition of input-to-output skip-connections. This leads us to believe that adding skip-connections within the LSTM-stack may help further improve model accuracy. Overall accuracy also improved by increasing the number of layers from 2 to 3. Lastly, observe that this sub-model is different from Xu et al. [31] wherein the three inputs are affine-transformed into D dimensions, summed and then passed through one fully-connected layer. After experimenting with their model we ultimately chose to instead feed the inputs (concatenated) to a fully-connected layer thereby allowing the MLP to naturally learn the input-to-output function. We also increased the number of layers to 3, changed activation function of hidden units from relu to tanh ¹⁰¹ and ensured that each layer had at least as many units as the softmax layer (K).

Table 5: Configuration of the Deep Output Layer MLP. $K = 339$ and 358 for I2L-140K and Im2latex-90k datasets respectively.

Layer	Num Units	Activation
3 (output)	K	softmax
2	$\max(358, K)$	tanh
1	$\max(358, K)$	tanh

A.5 Init model

The init model MLP is specified in Table 6. We questioned the need for the Init Model and experimented just using zero values for the initial state. That caused a slight but consistent decline ($< 1\%$) in the validation score, indicating that the initial state learnt by our Initial State Model did contribute in some way towards learning and generalization. Note however that our Init Model is different than 31, in that our version uses all L feature vectors of $\mathbf{a}$ while theirs takes the average. We also added a hidden layer and used tanh activation function instead of relu . We did start off with their version but that did not provide an appreciable impact to the bottom line (validation). This made us hypothesize that perhaps taking an average of the feature vectors was causing a loss of information; and we mitigated that by taking in all the L feature vectors without summing them. After making all these changes, the Init Model yields a consistent albeit small performance improvement (Table. 7). But given that it consumes ~ 7.5 million parameters, its usefulness remains in question.

Table 6: Init Model layers.

Layer	Num	Units	Activation Function
Output	2Q	n	tanh
Hidden	1	100	tanh

Table 7: Impact of the Init Model on overall performance. Since it comprises 10-12% of the total params, it may as well be omitted in exchange for a small performance hit.

Model	Init Model Present?	Validation BLEU	Num Params
I2L-NOPOOL	Yes	89.09%	7,569,300
I2L-NOPOOL	No	88.20%	0
I2L-STRIPS	Yes	89.00%	7,569,300
I2L-STRIPS	No	88.74%	0

¹⁰¹We changed from relu to tanh partly in order to remedy 'activation-explosions' which were causing floating-point overflow errors.

A.6 Training and dataset

A.6.1 Alpha penalty

Please see equations 13 through 13e. The loss function equation stated in the paper is Equation 13 but with λ_A set to 0. That was the case when training models whose results we have published, however at other times we had included a penalty term $\lambda_A \mathcal{A}$ which we discuss next. Observe that while $\sum_l \alpha_{t,l} = 1$, there is no constraint on how the attention is distributed across the L locations of the image. The term $\lambda_A \mathcal{A}$ serves to steer the variance of α_l by penalizing any deviation from a desired value. ASE (Alpha Squared Error) is the sum of squared-difference between α_l and its mean τ/L ; and ASE_N is its normalized value ¹³ $\in [0,100]$ ¹⁴. Therefore $ASE_N \propto ASE \propto \sigma_{\alpha_l}^2$. ASE_T which is the desired value of ASE_N , is a hyperparameter that needs to be discovered through experimentation¹⁵. Table 8 shows training results with alpha-penalty details.

Table 8: Training metrics. $\lambda_R = 0.00005$ and $\beta_2 = 0.9$ for all runs.

Dataset	Model	Init Model?	λ_A	β_1	Training Epochs	Training BLEU	Validation ED	ASE_N
I2L-140K	I2L-STRIPS	Yes	0.0	0.5	104	0.9361	0.0677	5.3827
	I2L-STRIPS	No	0.0	0.5	75	0.9300	0.0691	4.9899
	I2L-NOPOOL	Yes	0.0	0.5	104	0.9333	0.0684	4.5801
	I2L-NOPOOL	No	0.0	0.1	119	0.9348	0.0738	4.7099
Im2latex-90k	I2L-STRIPS	Yes	0.0	0.5	110	0.9366	0.0688	5.1237
	I2L-STRIPS	No	0.0005	0.5	161	0.9386	0.0750	4.8291

Default values of β_1 and β_2 of the ADAM optimizer - 0.9 and 0.99 - yielded very choppy validation score curves with frequent down-spikes where the validation score would fall to very low levels, ultimately resulting in lower peak scores. Reducing the first and second moments (i.e. β_1 and β_2) fixed the problem suggesting that the default momentum was too high for our ‘terrain’. We did not use dropout for regularization, however increasing the data-set size (I2L-140K) and raising the minimum-word-frequency threshold from 24 (Im2latex-90k) to 50 (I2L-140K) did yield better generalization and overall test scores (Table 8). Finally, normalizing the data¹⁶ yielded about 25% more accuracy than without.

$$\mathcal{J} = -\frac{1}{\tau} \log(P_r(\mathbf{y}|\mathbf{a})) + \lambda_R \mathcal{R} + \lambda_A \mathcal{A} \quad (13)$$

$$\mathcal{R} = \frac{1}{2} \sum_{\theta} \theta^2 \quad (13a)$$

$$\mathcal{A} = (ASE_N - ASE_T) \quad (13b)$$

$$ASE_N = \frac{100}{\tau^2 \left(\frac{L-1}{L}\right)} \cdot ASE \quad (13c)$$

$$ASE = \sum_{l=1}^L \left(\alpha_l - \frac{\tau}{L} \right)^2 \quad (13d)$$

$$\alpha_l := \sum_{t=1}^{\tau} \alpha_{t,l} \quad (13e)$$

¹³It can be shown that $\tau^2 \left(\frac{L-1}{L}\right)$ is the maximum possible value of ASE .

¹⁴We normalize ASE so that it may be compared across batches, runs and models.

¹⁵Start with $ASE_T = 0$, observe where ASE_N settles after training, then set ASE_T to that value and repeat until approximate convergence.

¹⁶Normalization was performed using the method and software used by [6] which parses the formulas into an AST and then converts them back to normalized sequences.